

통계이야기 (StatsStory)

AI 기반 국토교통부 통계 분석 시스템

발표일: 2025년 10월 17일

프로젝트 개요

프로젝트 시작 배경

실무 니즈에서 출발한 프로젝트

- 국토교통부 통계누리 사이트의 "통계이야기" 페이지
- 매달 카드뉴스 제작 업무 담당
 - 예: 8월 통계이야기 → 7월에 올라온 통계 중 선택

겪었던 문제점

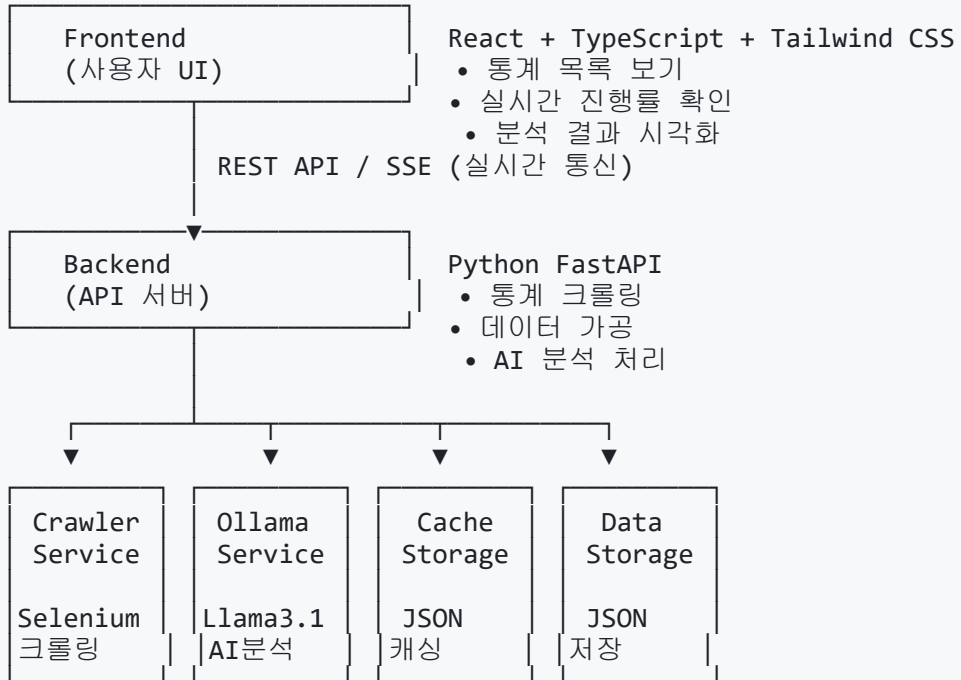
- ✗ 디자이너 배경이라 통계 분석 경험 부족
- ✗ 1달치 모든 통계를 일일이 확인하기 불편
- ✗ 어떤 통계를 선택할지 방향 잡기 어려움
- ✗ 통계표만 봐서는 기획 포인트 파악 힘들

해결 아이디어

 "AI 인사이트만 있어도 기획할 때 방향 잡기가 편하지 않을까?"

- 최신 통계 1달치만 자동 수집
- AI가 각 통계의 핵심 인사이트 자동 생성
- 통계별 특징을 한눈에 비교
- 어떤 통계로 카드뉴스를 만들지 결정

시스템 아키텍처



주요 서비스 역할:

- **Crawler Service:** Selenium 기반 웹 크롤링 (메타데이터 + 데이터 수집)
- **Ollama Service:** Llama 3.1 로컬 LLM으로 AI 인사이트 생성
- **Cache Storage:** 수집 결과 캐싱 (metadata, collected_data, analysis_results)
- **Data Storage:** 통계 데이터 JSON 저장 및 관리

기술 스택

Frontend 기술

- **React 19** - 최신 UI 프레임워크
- **TypeScript** - 타입 안정성 보장
- **Tailwind CSS** - 모던한 스타일링
- **Axios** - HTTP 클라이언트
- **Framer Motion** - 부드러운 애니메이션
- **Heroicons & Lucide React** - 아이콘 라이브러리

Backend 기술

- **FastAPI** - 고성능 Python 웹 프레임워크
- **Selenium** - 동적 웹 크롤링
- **BeautifulSoup4** - HTML 파싱
- **Pandas** - 데이터 처리 및 분석
- **Pydantic** - 데이터 검증 및 직렬화
- **Ollama (Llama 3.1)** - 로컬 AI 분석 엔진

배포 & 확장성

핵심 워크플로우

1 최신 통계 수집

- 국토교통부 통계누리에서 최근 1개월 통계 자동 크롤링
- 매달 "통계이야기" 제작을 위한 후보 통계 수집

2 사용자 통계 선택

- 관심 있는 통계를 클릭

3 실시간 데이터 수집 & 분석

- 메타데이터 수집 (통계 개요, 용어정의 등)
- 엑셀 파일 자동 다운로드 (우선)
- 테이블 데이터 직접 수집 (백업)
- 로컬 AI 기반 자동 분석

4 결과 확인 및 활용

- AI 인사이트로 통계별 특징 파악
- 어떤 통계로 카드뉴스를 만들지 빠르게 결정
- 구조화된 데이터로 추후 활용

주요 기능 - 통계 목록 화면

기능 특징

- ✓ 최근 1개월 통계 자동 표시
- ✓ 카테고리별 필터링 (국토계획, 교통, 주택 등)
- ✓ 수집 완료 통계 체크마크 표시
- ✓ 실시간 서버 상태 표시
- ✓ 반응형 카드 레이아웃

사용자 경험

- 직관적인 카드 디자인
- 한눈에 들어오는 통계 정보
- 원클릭으로 분석 시작

주요 기능 - 실시간 진행률 표시

SSE (Server-Sent Events) 기술

 서버 → 클라이언트 실시간 데이터 푸시

표시되는 정보

1. 현재 진행 단계 (메타데이터 수집, 테이블 수집 등)
2. 진행률 (%) - 프로그레스바로 시각화
3. 현재 작업 내용 - 상세 메시지 표시
4. 예상 소요 시간 - 사용자 대기 시간 안내
5. 취소 기능 - 언제든지 작업 중단 가능

장점

- ✧ 작업이 멈춘 것처럼 보이지 않음
- ✧ 투명한 프로세스 제공
- ✧ 사용자 이탈 방지



주요 기능 - 분석 결과 화면

제공되는 분석 결과

1. 기본 통계

- 평균, 중앙값, 최대값, 최소값
- 전체 데이터 포인트 수

2. 구조화된 데이터

- 연도별 원본 데이터 (JSON 형태)
- 테이블별 데이터 분리 저장
- 캐시를 통한 빠른 재사용

3. AI 인사이트 (텍스트 기반)

- 데이터 트렌드 분석
- 주요 변화 시점 탐지
- 통계표별 특징 분석
- 통계적 의미 해석

핵심 기술 - 동적 웹 크롤링

도전 과제

- ✗ 복잡한 **JavaScript** 기반 통계 사이트
- ✗ 동적 렌더링으로 일반 크롤링 불가능
- ✗ 복잡한 테이블 구조 (병합 셀, 다층 헤더)
- ✗ 다양한 데이터 형식 (IBSheet, 엑셀 등)

해결 방법 - 이중 전략

✓ 우선: 엑셀 다운로드 방식

- 브라우저 자동화로 엑셀 파일 다운로드
- Pandas로 파일 파싱 및 구조화

✓ 백업: 직접 테이블 수집

- Selenium 헤드리스 브라우저 사용
- JavaScript 완전 로딩 대기 후 DOM 파싱
- 테이블 구조 보존하며 JSON 변환

✓ 안정성 확보

- 모듈화된 크롤러 구조 (기능별 분리)

핵심 기술 - AI 분석 엔진

Ollama 로컬 LLM 활용

 오픈소스 로컬 AI로 데이터 프라이버시 보장

기술 스펙 & 제약사항

- 모델: Llama 3.1 (8B Instruct)
- 실행 환경: 로컬 서버 (CPU 전용, GPU 없음)
- 장점:
 - 비용 없음 (API 호출 비용 0원)
 - 데이터 외부 유출 없음
- 제약사항:
 - 16K 토큰 제한
 - CPU 성능 제한
 - Claude/GPT 대비 분석 품질 차이
- 극복 방법:
 - 통계표별 분할 분석으로 토큰 제한 회피
 - CPU 최적화 모델 사용

분석 프로세스

핵심 기술 - 캐시 시스템

다층 캐시 구조

```
backend/cache/  
├── metadata/      # 통계 메타데이터  
├── collected_data/ # 수집된 원본 데이터 (JSON)  
├── analysis_results/ # AI 분석 결과  
├── recent_stats/   # 최근 통계 목록  
└── tables/        # 테이블 구조 데이터
```

캐시 전략

- 한 번 수집한 데이터는 재사용
- 통계명 기반 캐시 키 생성
- 자동 캐시 유효성 검증

성능 개선 효과

- ⚡ 응답 속도: 초 단위 → 밀리초 단위
- ⚡ 불필요한 크롤링 방지
- ⚡ 서버 부하 감소

UI/UX 디자인 철학

디자인 시스템

- Tailwind CSS 기반 일관된 디자인
- 반응형 레이아웃 - 모바일/태블릿/데스크톱 대응
- 직관적인 색상 체계 - Primary Blue (#2563eb)
- 부드러운 애니메이션 - Framer Motion

사용자 경험 (UX)

✨ 직관적인 3단계 프로세스

선택 → 분석 → 결과 확인

✨ 실시간 피드백

로딩 상태, 진행률, 에러 메시지

✨ 데이터 중심 UI

텍스트 인사이트와 구조화된 데이터 표시

✨ 원활한 화면 전환

페이지 새로고침 없이 SPA 경험

📁 프로젝트 구조

```
statsStory/
├── backend/                                     # FastAPI 서버
│   ├── app/
│   │   ├── api/                               # API 라우터 (4개 모듈)
│   │   │   ├── health_router.py
│   │   │   ├── stats_manager.py
│   │   │   ├── analysis_router.py ★ 주요 분석 로직
│   │   │   └── data_router.py
│   │   ├── services/                         # 비즈니스 로직 (12개 서비스)
│   │   │   ├── crawler_service.py ★ 크롤링 총괄
│   │   │   ├── ollama_service.py ★ AI 인사이트 (실사용)
│   │   │   ├── progress_service.py ★ 실시간 진행률
│   │   │   ├── data_storage.py ★ 데이터 저장
│   │   │   ├── ai_service.py (미사용)
│   │   │   ├── mcp_client.py (미사용)
│   │   │   └── ...
│   │   ├── crawlers/                        # 크롤링 모듈
│   │   │   ├── modular_molit_crawler.py ★ 실사용
│   │   │   └── legacy/ (백업용)
│   │   ├── models/                          # 데이터 모델
│   │   ├── prompts/                        # AI 프롬프트 (미사용)
│   │   ├── cache/                          # 캐시 저장소 ★
│   │   │   ├── metadata/
│   │   │   ├── collected_data/
│   │   │   ├── analysis_results/
│   │   │   └── recent_stats/
│   │   └── main.py                          # FastAPI 앱 진입점
│   └── frontend/                            # React 앱
│       ├── src/
│       │   ├── components/                  # UI 컴포넌트 (10개)
│       │   │   ├── StatCard.tsx
│       │   │   ├── RealTimeProgressViewer.tsx ★ SSE
│       │   │   ├── EnhancedBasicStatisticsViewer.tsx
│       │   │   └── ...
│       │   ├── services/                   # API 클라이언트
│       │   │   ├── api.ts ★ API 호출 + SSE
│       │   └── App.tsx                     # 메인 앱 컴포넌트
│       └── mcp-servers/                    # (향후 확장용, 현재 미사용)
│           ├── pandas_analysis_server.py
│           └── visualization_server.py
```

★ = 실제 사용 중인 핵심 파일

개발 방법론 - 바이브 코딩

개발 규모

- 개발 기간: 2-3주 (40-60시간)
- 개발 인원: 1명 (디자이너 배경)
- 작업 시간: 주말 + 퇴근 후 2-3시간
- 산출물: 일자별 개발 일지 20여 개 (docs 폴더)

바이브 코딩이란?

AI와 대화하며 코드를 작성하는 방식

- Claude (Cursor AI)와 실시간 대화
- 문제 설명 → AI가 코드 제안 → 테스트 → 개선
- 전통적인 검색 & 복붙이 아닌 협업 개발

바이브 코딩의 장점

✓ 비개발자도 복잡한 시스템 구축 가능

- 디자이너가 풀스택 개발 완성
- 전통적 방식 대비 2-3배 빠름

✓ 프론트엔드 개발 프로세스

- 와이어프레임 손그림 → Claude에게 설명

개발 과정 - 주요 챌린지

1. IBSheet 동적 테이블 크롤링

문제점

- 특수 JavaScript 라이브러리로 렌더링
- DOM에 직접 접근 불가

해결책

- Selenium으로 완전 로딩 대기
- JavaScript 실행 완료 후 DOM 파싱

2. 대용량 데이터 처리

문제점

- 수십 년치 통계 데이터
- 메모리 부족 위험

해결책

- 캐시 시스템 구축
- 점진적 로딩 (Lazy Loading)
- Pandas 최적화

개발 과정 - 주요 챌린지 (계속)

3. 실시간 진행률 표시

문제점

- 긴 작업 시간 (1-2분)
- 사용자 이탈 가능성

해결책

- SSE (Server-Sent Events) 도입
- 단계별 진행률 실시간 전송
- 취소 기능 제공

4. AI 분석 - 하드웨어 제약 극복

하드웨어 환경

- CPU만 있는 노트북 (GPU 없음)
- Claude/GPT 수준의 분석 어려움
- 메모리 및 토큰 제한

문제점

- 처음 시도: 통계 전체 데이터를 한 번에 분석
- 결과: 토큰 양 초과로 실패 (16K 제한)
- Claude/GPT API 사용 고려했으나 비용 부담

향후 개선 방향

단기 목표 (3개월)

성능 개선 ★ 최우선

- GPU 활용 (CUDA 지원)
- 경량화 모델 테스트
- 캐싱 전략 강화
- 배치 처리 최적화

데이터 시각화 구현

- HTML 기반 차트 생성 (Chart.js, D3.js)
- 카드뉴스 형태 레이아웃 설계
- 인사이트 + 차트 통합 뷰

더 많은 통계 카테고리 지원

데이터 비교 기능 (통계 간 비교)

중기 목표 (6개월)

카드뉴스 완전 구현 ★

- 시각화된 분석 결과 제공



프로젝트 성과

기술적 성과

- 완전 자동화된 크롤링 시스템 구축
- 실시간 진행률 표시 (SSE) 구현
- 로컬 AI 기반 분석 엔진 통합
- 안정적인 캐시 시스템 구축
- 데이터 구조화 및 저장 시스템 완성
- 토큰 제한 문제 해결 - 통계표별 분할 전략

사용자 경험 개선

- 자동 수집 및 인사이트 생성 (통계당 5-10분)
- 직관적인 UI/UX 제공
- 실시간 진행률 피드백으로 투명성 확보
- CPU만으로 동작 - GPU 불필요

개발 경험

- 비개발자(디자이너)가 풀스택 개발 완성
- 바이브 코딩으로 2-3주 만에 구현 (40-60시간)
- 코드 품질: 모듈화된 구조, 타입 안정성, 에러 핸들링
- 문서화: 일자별 개발 일지 20여 개 작성

핵심 가치 제안

1 실무자 관점의 실용성

"통계이야기" 제작자를 위한 실용 도구

- 통계 분석 전문 지식 불필요
- AI가 자동으로 핵심 인사이트 추출
- 어떤 통계를 선택할지 의사결정 지원

2 시간 배분 최적화

기존: 통계 하나하나 수동 분석

개선: 밤사이 자동 분석 → 아침에 결과 확인

- 실제 소요 시간: 통계당 5-10분
- 하지만 사람이 직접 작업할 필요 없음

3 기획 효율성 향상

- AI 인사이트로 기획 방향 빠르게 수립
- 통계별 핵심 포인트 한눈에 비교
- 카드뉴스 소재 선정 시간 단축

4 하자 가는 선

⚠ 현재 한계점 & 개선 계획

주요 한계점

1. 성능 문제 (가장 큰 이슈)

- ✗ 통계표당 3~10분 소요 (CPU 환경)
- ✗ 대형 통계(10개 이상 표)는 1시간 이상 소요
- ✗ 실시간 사용 불가능

2. 분석 품질

- ✗ Claude/GPT 대비 인사이트 품질 차이
- ✗ 복잡한 통계 해석 한계
- ✗ 한국어 이해도 제한적

3. 사용성

- ✗ 밤사이 배치 처리 필요
- ✗ 즉각적인 피드백 어려움

개선 계획

✅ 단기 (1-2개월)

- GPU 서버 확보 또는 클라우드 GPU 활용
- 모델 양자화 및 최적화
- 병렬 처리 개선

✅ 중기 (3-6개월)

- 더 효율적인 경량 모델 테스트

배운 점 & 인사이트

기술적 학습

- 바이브 코딩 - AI(Claude)와 대화하며 개발
- 동적 웹 크롤링 노하우 습득
- **FastAPI + React** 풀스택 개발 경험
- **SSE** 실시간 통신 구현 경험
- 로컬 LLM 통합 및 제약사항 극복 (Ollama)
- 엑셀 자동화 및 파싱 기술
- 토큰 제한 문제 해결 - 데이터 분할 전략

프로젝트 관리

- 점진적 개선 중요성 (MVP → 기능 추가)
- 사용자 피드백 반영의 중요성
- 문서화의 중요성 (개발 과정 기록)

비즈니스 인사이트

- 실무 문제에서 출발한 프로젝트의 가치
- 공공 데이터의 활용 가능성 무한함

시연 (Live Demo)

실제 시스템 시연

1. 통계 목록 확인

- 최근 1개월 통계 자동 표시
- 카테고리 필터링

2. 통계 선택 및 분석 시작

- 원클릭으로 분석 시작

3. 실시간 진행률 확인

- SSE로 단계별 진행 상황 표시

4. 분석 결과 확인

- 기본 통계량
- 구조화된 데이터 (테이블)
- AI 생성 인사이트 (텍스트)

? Q&A

자주 묻는 질문

Q1. 왜 이 프로젝트를 만들게 되었나요?

A. 매달 "통계이야기" 카드뉴스를 제작해야 하는데, 1달치 통계를 일일이 확인하고 분석하는 게 너무 불편했습니다. 디자이너 배경이라 통계 분석 경험도 없어서, AI가 인사이트만 뽑아줘도 기획 방향을 잡기 훨씬 수월할 것 같아 시작했습니다.

Q2. 실제로 업무에 활용하고 있나요?

A. 현재 1단계(데이터 수집 + 인사이트)까지 완료했습니다. 다만 통계당 5-10분 걸려서 실시간 활용은 어렵고, 전날 밤 배치로 돌려놓고 아침에 결과를 확인하는 방식으로 활용 중입니다. 성능 개선이 필요한 상태입니다.

Q3. 크롤링이 불법이 아닌가요?

A. 공공데이터는 개방 원칙에 따라 활용 가능합니다. 서버에 부담을 주지 않도록 적절한 대기 시간을 두고 있습니다.

Q4. AI 분석 비용은 얼마나 드나요?

A. Ollama 로컬 LLM을 사용하여 API 비용이 전혀 들지 않습니다. GPU 없이 CPU만으로도 동작하지만, 통계표당 3-10분 소요되어 성능이 느린 것이 단점입니다. GPU를 활용하면 훨씬 빨라질 것으로 예상됩니다.

Q4-1. 왜 Claude나 GPT API를 안 쓰나요?

A. 초기엔 고려했지만 지속적인 비용 부담이 있고, CPU 노트북에서도 동작하는 완전 로컬 솔루션을 만들고 싶었습니다. 품질 차이는 있지만 업무에는 충분히 활용 가능한 수준입니다.

Q5. 다른 사람도 사용할 수 있나요?

A. 현재는 개인 프로젝트지만, 같은 업무를 하는 동료들에게도 유용할 것으로 생각됩니다. 향후 다른 기관 통계로 확장하면 더 많은 실무자들이 활용 가능할 것입니다.

Q6. 개발 경험이 없는데 어떻게 만들었나요?

A. 바이브 코딩으로 개발했습니다. Claude AI(Cursor)와 대화하며 "이런 기능을 만들고 싶다"고 설명하면 AI가 코드를 제안하고, 테스트하면서 개선해나갔습니다. 물론 완전 자동은 아니고 AI 제안을 이해하고 디버깅하는 과정이 필요했습니다.

참고: "통계이야기"란?

국토교통부 통계누리의 콘텐츠

통계이야기 페이지

- 매달 발행되는 통계 카드뉴스
- 일반인이 쉽게 이해할 수 있도록 시각화
- 최근 발표된 통계 중 주목할 만한 것 선정

제작 과정 (기존)

1. 전월 발표 통계 **전체 확인** (20-30개)
2. 각 통계표 상세 분석 및 검토
3. 카드뉴스 소재 선정
4. 기획 및 디자인
5. 최종 발행

제작 과정 (이 시스템 활용 시)

1.  전날 밤 자동 수집 및 인사이트 생성 예약
 - 통계당 5-10분 소요 (20-30개 → 총 2-5시간)
 - 사람이 대기할 필요 없음 (배치 작업)

감사합니다! 🙏

프로젝트 정보

- **GitHub:** [프로젝트 링크 추가]
- **이메일:** [이메일 추가]

추가 질문이 있으시면 언제든지 말씀해주세요!

부록: 기술 상세 스펙

개발 환경

- 개발 기간: 2-3주 (40-60시간)
- 개발 인원: 1명
- 개발 방식: 바이브 코딩 (Claude AI 협업)

실행 환경

- **Node.js**: v16+
- **Python**: 3.9+
- **Chrome/Chromium**: Latest (Selenium용)
- **Ollama**: Latest (AI 분석용)

배포 현황

- 현재: 로컬 환경에서만 실행
- 향후: Docker Compose 원클릭 설치 계획
- 폐쇄망: Ollama 로컬 LLM이라 가능 (모델 사전 다운로드 필요)

주요 API 엔드포인트

실제 사용 중:

- GET /api/recent-stats - 최신 통계 목록 조회

부록: 프로젝트 회고

프로젝트를 통해 얻은 것

기술적 성장

- 비개발자에서 풀스택 시스템 구축
- 실전 크롤링, AI 통합, 실시간 통신 경험
- 바이브 코딩으로 생산성 극대화

문제 해결 능력

- 하드웨어 제약 (CPU) → 분할 전략
- 토큰 제한 → 통계표별 처리
- 성능 이슈 → 배치 처리로 전환

실무 적용

- 개인 불편 → 실용 도구
- MVP로 가치 검증
- 향후 발전 가능성 확인

교훈

- 완벽하지 않아도 시작하기
- 한계를 인정하고 창의적 해결
- 문서화의 중요성